



BÀI THỰC HÀNH 6

XÂY DỰNG FRONTEND VỚI REACTJS (tt)

Bài 1: Thêm và Sửa Review.

1.1 Tạo login component.

Yêu cầu khi thiết lập login component thì khi người dùng đăng nhập thành công, họ sẽ thấy được các chức năng như Edit và Delete review của chính họ.

Reviews

ie213xyz reviewd on 18th April 2022
nice!

ie213kaka reviewd on 18th April 2022
great!

ie213khoa reviewd on 18th April 2022
bad!

[Edit](#) [Delete](#)

ie213khoa reviewd on 18th April 2022
bad bad!

[Edit](#) [Delete](#)

Sau khi login thành công, người dùng sẽ được redirect về lại trang Home.

Trong login.js:

```
import React, { useState } from 'react'
import Form from 'react-bootstrap/Form';
import Button from 'react-bootstrap/Button';
const Login = props => {
  const [name, setName] = useState('');
  const [id, setId] = useState('');
  const onChangeName = e => {
    const name = e.target.value;
    setName(name);
  }
  const onChangeId = e => {
    const id = e.target.value;
    setId(id);
  }
  const login = () => {
    props.login({ name: name, id: id });
    props.history.push('/');
  }
}
```



```
    return (  
      <div>  
        <Form>  
          <Form.Group>  
            <Form.Label>Username</Form.Label>  
            <Form.Control  
              type="text"  
              placeholder="Enter username"  
              value={name}  
              onChange={onChangeName}  
            />  
          </Form.Group>  
          <Form.Group>  
            <Form.Label>ID</Form.Label>  
            <Form.Control  
              type="text"  
              placeholder="Enter id"  
              value={id}  
              onChange={onChangeId}  
            />  
          </Form.Group>  
          <Button variant="primary" onClick={login}>  
            Submit  
          </Button>  
        </Form>  
      </div>  
    )  
  }  
  export default Login;
```

Thêm xử lý login() trong App.js:

```
<Route path="/login" render={(props) =>  
  <Login {...props} login={login} />  
}>  
</Route>
```

1.2 Thêm review

Tạo các biến như hướng dẫn sau:

- Biến editing sẽ có giá trị true khi component đang ở chế độ Editing.
- Ngược lại là chế độ thêm review.
- Biến trạng thái review được thiết lập thông qua biến initialReviewState.
- Trong chế độ editing, initialReviewState sẽ được thiết lập có nội dung text.
- Biến trạng thái submitted để theo dấu nếu như có một review được thêm mới.

Tạo các hàm phù hợp:

- Hàm onChangeReview() theo vết khi người dùng thêm giá trị review dưới form.
- Hàm saveReview() được gọi khi nút submit được nhấn.



- Trong hàm này, đầu tiên ta tạo một object tên data chứa các giá trị thuộc tính của review.
- 2 giá trị name và user_id sẽ nhận từ props được gửi từ App.js.
- Lấy movie_id trực tiếp từ url (xem lại nội dung movie.js).
- Sau đó gọi hàm createReview(data) trong MovieDataService.
- Định tuyến này gọi tới ReviewsController trong backend và gọi apiPostReview(), trích xuất data từ request's body params.

Phần return() chứa nội dung JSX giúp hiển thị và xử lý tính năng thêm review:

Movie Reviews [Movies](#) [Logout User](#)

Create Review

Submit



```
import React, { useState } from 'react';
import MovieDataService from "../services/movies";
import { Link } from "react-router-dom";
import Form from 'react-bootstrap/Form';
import Button from 'react-bootstrap/Button';

const AddReview = props => {
  let editing = false;
  let initialReviewState = "";
  const [review, setReview] = useState(initialReviewState);
  // keeps track if review is submitted
  const [submitted, setSubmitted] = useState(false);
  const onChangeReview = e => {
    const review = e.target.value;
    setReview(review);
  }
  const saveReview = () => {
    var data = {
      review: review,
      name: props.user.name,
      user_id: props.user.id,
      movie_id: props.match.params.id // get movie id direct from url
    }
    MovieDataService.createReview(data)
      .then(response => {
        setSubmitted(true);
      })
      .catch(e => {
        console.log(e);
      })
  }
}
```



```
return (
  <div>
    {submitted ? (
      <div>
        <h4>Review submitted successfully</h4>
        <Link to={"/movies/" + props.match.params.id}>
          Back to Movie
        </Link>
      </div>
    ) : (
      <Form>
        <Form.Group>
          <Form.Label>{editing ? "Edit" : "Create"}
            Review</Form.Label>
          <Form.Control
            type="text"
            required
            value={review}
            onChange={onChangeReview}
          />
        </Form.Group>
        <Button variant="primary" onClick={saveReview}>
          Submit
        </Button>
      </Form>
    )}
  </div>
);
}

export default AddReview;
```

1.3 Sửa review

Viết mã nguồn thực hiện các việc sau:

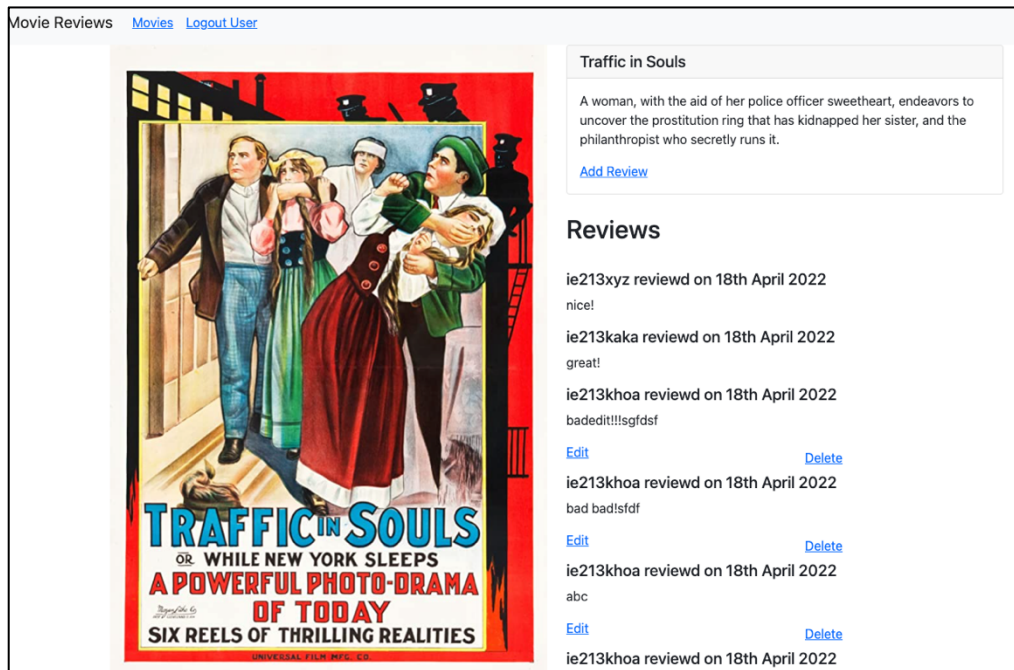
- Đầu tiên, kiểm tra trạng thái truyền vào cho AddReview (xem lại tệp tin movie.js sẽ thấy prop state).
- Nếu state được truyền vào chứa thuộc tính currentReview thì chuyển editing thành true và initialReviewState thành currentReview.review.
- Nếu editing là true thì gọi updateReview() trong MovieDataService.
- Phương thức apiUpdateReview() trong ReviewsController ở backend sẽ được gọi, tương tự apiPostReview.



```
const AddReview = props => {
  let editing = false;
  let initialReviewState = '';

  if (props.location.state && props.location.state.currentReview) {
    editing = true;
    initialReviewState = props.location.state.currentReview.review;
  }
  ...
  const saveReview = () => {
    var data = {
      ...
    }
    if (editing) {
      // get existing review id
      data.review_id = props.location.state.currentReview._id;
      MovieDataService.updateReview(data)
        .then(response => {
          setSubmitted(true);
          console.log(response.data);
        })
        .catch(e => {
          console.log(e);
        });
    }
    else {
      MovieDataService.createReview(data)
        .then(response => {
          setSubmitted(true);
        })
        .catch(e => {
          console.log(e);
        });
    }
  }
  return (
```

Chạy ứng dụng sẽ cho phép cập nhật lại review



Bài 2: Xóa review

Thêm mã nguồn vào movie.js để xử lý phần xóa review:

- Trong nút delete, ta truyền review id và index chúng ta nhận được từ movie.reviews.map() vào phương thức deleteReview().

```
<Col>  
<Button variant="link" onClick={() => deleteReview(review._id,  
index)}>  
Delete  
</Button>  
</Col>
```

- Trong phương thức deleteReview(), ta gọi hàm deleteReview() trong MovieDataService.
- Sau đó, tạo một callback .then() sẽ được gọi sau khi deleteReview() được gọi xong.
- Trong callback này, ta lấy mảng các reviews trong trạng thái hiện tại. Sau đó cung cấp index của review sẽ bị xóa cho phương thức splice() để xóa nó đi.
- Cuối cùng là cập nhật lại mảng reviews như là trạng thái mới.



```
const deleteReview = (reviewId, index) => {
  MovieDataService.deleteReview(reviewId, props.user.id)
    .then(response => {
      setMovie((prevState) => {
        prevState.reviews.splice(index, 1);
        return({...prevState});
      });
    })
    .catch(e => {
      console.log(e);
    });
  }
  return (
```

Chạy lại ứng dụng -> log in -> chọn một movie cụ thể có review mà mình đã đăng.

Bài 3: Lấy dữ liệu cho trang tiếp theo

3.1 getAll()

Trong component movies-list.js thêm mã nguồn để thực hiện yêu cầu.

- Thêm 2 biến trạng thái currentPage và entriesPerPage.
- Thiết lập 2 biến trạng thái này trong phương thức retrieveMovies().
- Thêm một useEffect hook:

```
useEffect(() => {
  retrieveMovies();
}, [currentPage]);
```

- Biến trạng thái currentPage được truyền trong tham số thứ 2 (trong mảng) nhằm mục đích khi giá trị của nó thay đổi thì hàm retrieveMovies() sẽ được gọi.

- Quan trọng: nhớ truyền currentPage vào lời gọi MovieDataService.get().
- Đồng thời, thêm đoạn mã xử lý vào hàm return().



```
const [movies, setMovies] = useState([]);
const [searchTitle, setSearchTitle] = useState("");
const [searchRating, setSearchRating] = useState("");
const [ratings, setRatings] = useState(["All Ratings"]);

const [currentPage, setCurrentPage] = useState(0);
const [entriesPerPage, setEntriesPerPage] = useState(0);

useEffect(() => {
  retrieveMovies();
  retrieveRatings();
}, []);

useEffect(() => {
  retrieveMovies();
}, [currentPage]);

const retrieveMovies = () => {
  MovieDataService.getAll(currentPage)
    .then(response => {
      console.log(response.data);
      setMovies(response.data.movies);
      setCurrentPage(response.data.page);
      setEntriesPerPage(response.data.entries_per_page);
    })
    .catch(e => {
      console.log(e);
    });
}
```

```
<br />
Showing page: {currentPage}.
<Button
  variant="link"
  onClick={() => {setCurrentPage(currentPage + 1)}}>
  Get next {entriesPerPage} results
</Button>
```

3.2 find()

Trong movies-list.js ta điều chỉnh mã nguồn theo các yêu cầu sau:

- Đầu tiên, tạo biến trạng thái currentSearchMode để nhận 2 giá trị “findByTitle” hoặc “findByRating”.
- Sử dụng một useEffect() để khi nào currentSearchMode thay đổi thì thiết lập lại currentPage về 0.



- Tạo phương thức mới `retrieveNextPage()` -> dựa vào `currentSearchMode` để gọi các hàm tương ứng.
- Thêm tham số `currentPage` vào trong lời gọi `MovieDataService.find()`.

```
const [entriesPerPage, setEntriesPerPage] = useState(0);
const [currentSearchMode, setCurrentSearchMode] = useState("");
...
useEffect(() => {
    setCurrentPage(0);
}, [currentSearchMode]);
useEffect(() => {
    //retrieveMovies();
    retrieveNextPage();
}, [currentPage]);

const retrieveNextPage = () => {
    if(currentSearchMode === "findByTitle")
        findByTitle();
    else if(setCurrentSearchMode === "findByRating")
        findByRating();
    else
        retrieveMovies();
}
...
const find = (query, by) => {
    MovieDataService.find(query, by, currentPage)
        .then(response => {
            console.log(response.data);
            setMovies(response.data.movies);
        })
        .catch(e => {
            console.log(e);
        });
}
```

- Lần lượt thêm các dòng mã lệnh `setCurrentSearchMode()` tương ứng vào 3 phương thức điều khiển:
 - `retrieveMovies()`;
 - `findByTitle()`;
 - `findByRating()`;



```
const retrieveMovies = () => {
  setCurrentSearchMode("");
  MovieDataService.getAll(currentPage)
    .then(response => {
      console.log(response.data);
      setMovies(response.data.movies);
      setCurrentPage(response.data.page);
      setEntriesPerPage(response.data.entries_per_page);
    })
    .catch(e => {
      console.log(e);
    });
}

...
const findByTitle = () => {
  setCurrentSearchMode("findByTitle");
  find(searchTitle, "title");
}
const findByRating = () => {
  setCurrentSearchMode("findByRating");
  if(searchRating === "All Ratings") {
    retrieveMovies();
  }
  else {
    find(searchRating, "rated");
  }
}
```